ConsignTrack

Source of Truth - Complete System Reference

A consignment operations platform that lets product owners (consignors) place goods with partner stores (consignees), and gives everyone one shared, auditable record of what was shipped, received, sold, owed, and paid.

Live app	`https://consigntrack-zeta.vercel.app`
Stack	Next.js 14 (App Router) - Supabase/PostgreSQL - Vercel
Document date	12 November 2026
Status	Authoritative reference for product, data model, and operations

1. What ConsignTrack Is

ConsignTrack digitises the consignment business model. In consignment, a **consignor** (the maker or brand that owns the goods) places inventory in a **consignee** (a partner store) without selling it outright. The store displays and sells the goods, keeps an agreed commission on each sale, and periodically settles the rest back to the consignor. Ownership stays with the consignor until an item is sold.

The hard part of running this by hand is trust and reconciliation: who holds how much stock, what actually sold, what is owed this period, and whether a payment was really sent and received. ConsignTrack replaces spreadsheets and group chats with a single shared system of record for all three parties.

Three parties, one record. An administrator provisions accounts; consignors manage their catalog and shipments; stores receive stock, sell by scanning barcodes, and settle. Every movement of stock and money is written to an append-only ledger that all sides read from, so the numbers always agree.

What the system guarantees

- **No public sign-up.** Accounts are issued by an administrator only, so every login is accountable.
- **One source of stock truth.** On-hand quantities are derived from an append-only ledger, never hand-edited.
- **Barcode-driven sales.** Items enter a sale only by scanning their barcode, which reduces mis-keyed sales.
- **Auditable settlements.** Every statement shows its gross, commission, and net, and every payment carries a reference and a two-sided confirmation.
- **Role isolation.** Database row-level security means a store only ever sees its own data, and a consignor only its own.

2. Roles & Access Model

ConsignTrack has four roles. Access is enforced in the database (row-level security), not just in the interface, so the boundaries hold even for direct data access.

Role	Who they are	What they can do
Administrator	The platform operator	Issue and terminate consignor and store accounts; set up and terminate agreements; set per-product commission overrides; view exceptions. No public sign-up exists.
Consignor	The product owner / brand	Manage their product catalog and SKUs, hold warehouse stock, dispatch shipments to stores, view what each store holds and owes, edit their own agreement terms, and confirm settlement payments received.
Store manager (consignee)	The partner retailer	Receive shipments, manage shelf inventory, record sales by scanning, raise return shipments, generate settlement statements and pay them.
Store staff	A store's floor staff	A narrower store login for day-to-day selling under a store.

Provisioning. The administrator creates each account on the Accounts screen. Issuing a store account also connects it to a consignor and opens their agreement in one step, so the consignor's products appear on the store's side immediately.

Workspaces (navigation)

Admin workspace	Consignor workspace	Store workspace
Overview	Dashboard	Dashboard
Accounts	Catalog / Products	Deliveries & Receiving
Agreements & commissions	Shipments / Deliveries	Inventory & Catalog
Exceptions	Stores & Agreements (Partners)	Sales (scan terminal)
Items	Settlements	Returns
	Insights / Analytics	Partners & Settlements
	Scan hub	Scan hub

3. Core Concepts & Glossary

Term	Meaning
Consignor	The owner of the goods (maker/brand). Holds warehouse stock and dispatches it to stores.
Consignee / Store	The partner retailer that receives, displays, and sells the goods on consignment.
Agreement	The commercial contract between one consignor and one store: commission %, settlement cadence, damage liability, consignment period, and end-of-period handling.
Commission	The store's share of each sale. Can be overridden per product.
SKU	Stock-keeping unit: a product's code (e.g., DAL-0001). Every product is auto-issued a SKU and a Code 128 barcode.
Serialized piece	An individual physical unit with its own unique barcode (serial format -NNNN). Scanning it sells that exact piece. (Model B.)
Warehouse stock	Quantity the consignor holds before dispatch (consignor-side ledger).
Shelf / store stock	Quantity a store holds on hand after receiving a shipment (store-side ledger).
Package / Shipment	A tracked movement of goods: OUTBOUND (consignor to store) or a RETURN (store to consignor).
Dispute	A quantity discrepancy raised at receiving; the difference is held aside (DISPUTE_HOLD) until resolved.
Settlement / Statement	A per-period reckoning for one agreement: gross sales minus commission (and returns/damage) equals net payable.
Payment / Cashless rail	The demo settlement payment: records method and a reference, then is confirmed by the consignor. No real funds move.
Append-only ledger	Stock and sales are recorded as immutable entries; corrections are new reversing entries, never edits.

4. Data Model & The Two-Tier Ledger

ConsignTrack keeps two separate stock ledgers so warehouse stock and store stock never get confused. On-hand numbers are always computed from these ledgers; they are never stored as a mutable counter.

Stock ledgers

Ledger	Scope	Entry types	On-hand via
warehouse_entries	Consignor warehouse	PRODUCE (intake +), DISPATCH (-), RETURN_IN (+)	get_warehouse_on_hand / v_warehouse_on_hand
ledger_entries	Store shelf	RECEIVE (+), SALE (-), DISPUTE_HOLD (-), adjustments	get_on_hand / v_on_hand ; v_in_dispute

Both ledgers are **append-only**: a database trigger blocks any UPDATE or DELETE and requires a reversing entry instead. This is what makes the history trustworthy and auditable.

Serialized pieces (Model B)

On top of the quantity ledger, each physical unit can be given a unique barcode and tracked individually in **product_units** (serial, status in_stock/sold, store, sold_at, sold_by). Minting creates the serials (format -NNNN); scanning a serial at the till sells that exact piece. This adds a tamper-evident layer over the aggregate counts.

Key tables

Table	Holds
consignors / stores	The two kinds of business party.
app_users	Maps an auth sign-in to a role and to its consignor or store.
agreements	Commercial terms per consignor-store pair (commission, cadence, liability, period, end-of-period, active).
commission_overrides	Per-product commission that overrides the agreement default.
products	Catalog: name, SKU, price, unit of measure, category, status.
product_units	Individual serialized pieces and their sold/unsold state.
packages / package_lines	Shipments and their line items (outbound and return).
settlements / settlement_lines	Period statements and their per-product breakdown.
payments	Settlement payments: method, amount, reference, paid/confirmed timestamps.

5. End-to-End Workflows

5.1 Provision accounts (Admin)

- Admin opens Accounts and issues a consignor (business name, contact, sign-in email, temporary password).
- Admin issues a store, choosing the connected consignor and the agreement terms (commission %, cadence, period). This creates the store sign-in and opens the agreement in one step.
- Issued accounts are listed; any can be Terminated (two-click confirm), which revokes the sign-in and deactivates its agreements while preserving history.

5.2 Build catalog & issue barcodes (Consignor)

- Add a product via a popup (name, category, variant, unit, price, opening stock). A SKU and Code 128 barcode are issued automatically.
- Opening stock is added to the warehouse ledger; individual pieces can be minted with serialized barcodes.
- Printable label sheets render the barcodes for sticking onto stock.

5.3 Dispatch and receive stock

- Consignor builds an OUTBOUND package (pick a store, add products with +/- steppers), which must fit available warehouse stock, then dispatches it (warehouse stock is deducted; package goes IN_TRANSIT).
- Store receives the package, entering received quantities. Received quantity posts RECEIVE (+) to the store ledger; any shortfall is raised as a dispute (DISPUTE_HOLD) and held until resolved.

5.4 Record sales by scanning (Store)

- The Sales terminal opens a full-page scanner. Each valid scan is ADDED TO A CART - a known SKU adds a quantity line; a serialized piece code adds that exact piece. Unrecognized codes are rejected with an indicator.
- Items enter the cart only by scanning (no click-to-add). After 3 failed scans a typed-code fallback appears.
- Pressing Record sale checks out the whole cart in ONE transaction, so it appears in Sales Records as a single grouped sale with a per-product breakdown. Pieces are marked sold only on a successful checkout.

5.5 Returns

- A store can build a RETURN package of unsold stock back to the consignor; the consignor receives it back into the warehouse ledger.

5.6 Settlement & the cashless payment rail

- A statement is generated for an agreement and period: gross sales, minus commission (and any returns/damage), equals net payable, with a per-product breakdown.
- States: pending -> paid / partial (store sends payment) -> confirmed / settled (consignor confirms receipt).
- The store pays via the cashless dialog (GCash / Maya / InstaPay / Bank transfer). The pay_settlement function validates the amount, records a payment row, synthesizes a reference (CT-XXXXXXXX), and recomputes the status. It is a DEMO rail - a reference is recorded, no real funds move.
- The consignor then confirms receipt, moving the statement to Settled.

6. Security & Integrity Model

- **Admin-only provisioning.** There is no public sign-up; accounts are created by the administrator through SECURITY DEFINER functions (admin_create_consignor, admin_issue_store_account).
- **Row-level security (RLS).** Every table is scoped by the signed-in identity via helper functions is_admin(), current_consignor_id(), and current_store_id() (all based on auth.uid()). A store cannot read another store's rows; a consignor cannot read another consignor's.
- **Trusted business logic.** State changes (sales, dispatch, receive, settle, pay) run inside SECURITY DEFINER PL/pgSQL functions that enforce the rules (authorization, oversell checks, amount limits) regardless of who calls them.
- **Append-only ledgers.** A trigger blocks edits/deletes on the stock ledgers; corrections are reversing entries. The sales history and on-hand numbers therefore cannot be silently rewritten.
- **Two-sided money.** A settlement needs the store to pay AND the consignor to confirm before it is Settled, so neither side can unilaterally close a statement.
- **Account & agreement termination.** Terminating an account bans its sign-in and drops its role mapping; terminating an agreement deactivates just that pairing. History is preserved in both cases.

Representative server functions (RPCs)

Function	Purpose
admin_create_consignor / admin_issue_store_account	Provision accounts (auth user + role mapping + business record + agreement).
admin_terminate_account	Ban sign-in, drop role mapping, deactivate agreements.
warehouse_intake	Add consignor warehouse stock (PRODUCE).
create_outbound_package / dispatch_package / receive_package	Build, send, and receive a shipment.
mint_units / sell_unit / lookup_unit	Create serialized pieces, sell one by barcode, validate one without selling.
pos_sale / checkout_sale	Record a sale; checkout_sale posts SKU lines + scanned pieces in one transaction so a cart groups as one sale.
generate_settlement_ledger	Produce a period statement from sales.
pay_settlement / confirm_payment_received	Record a cashless payment and confirm its receipt.
update_agreement_terms	Let the agreement's own consignor/store (or admin) edit terms.

7. Technical Architecture

Layer	Choice & notes
Framework	Next.js 14 (App Router) with React Server Components and Server Actions; TypeScript.
UI	Tailwind CSS with oklch design tokens; balanced blue+white branding; responsive down to phone width.
Data / Auth	Supabase (PostgreSQL + Auth). Row-level security on all tables; business logic in SECURITY DEFINER PL/pgSQL functions.
Barcodes	Code 128 generated with JsBarcode; camera scanning with html5-qrcode (SKU and serialized-piece codes).
Hosting	Vercel, with functions pinned to the Singapore region (sin1) next to the Supabase database (ap-southeast-1) for low latency (~40-60 ms TTFB).
Time	All dates render in Asia/Manila; the server runs in UTC and converts for display (phDateKey / phMonthDay helpers).
Schema history	35 numbered SQL migrations under supabase/migrations capture every schema change.

Recent schema milestones. 0033 added ml/oz units and warehouse intake; 0034 added serialized product units (Model B); 0035 let consignors/stores edit their own agreement; 0036 added piece look-up for the cart; 0037 added one-transaction checkout (grouped sales); 0038 added account termination.

8. Screen Reference (Routes)

Area	Route	Purpose
Auth	/login	Email + password sign-in (admin-provisioned accounts only).
Admin	/admin	Overview.
Admin	/admin/accounts	Issue consignor/store accounts; terminate accounts.
Admin	/admin/agreements	Create/edit/terminate agreements; per-product commission overrides.
Admin	/admin/exceptions, /admin/items	Exceptions queue and item lookup.
Consignor	/consignor	Dashboard.
Consignor	/consignor/products, /consignor/items	Catalog and item detail.
Consignor	/consignor/shipments, /consignor/deliveries	Build and track outbound shipments.
Consignor	/consignor/partners, /consignor/stores/[id]	Stores & agreements; per-store view.
Consignor	/consignor/settlements	Statements; confirm payments received.
Consignor	/consignor/analytics, /consignor/reports	Insights and reports.
Consignor	/consignor/scan	Scan hub.
Store	/store	Dashboard.
Store	/store/deliveries	Receive shipments; raise disputes.
Store	/store/inventory, /store/catalog	Shelf inventory and catalog.
Store	/store/sales, /store/sales/import	Scan-to-cart terminal; sales records; import.
Store	/store/returns	Build return shipments.
Store	/store/settlements, /store/partners	Generate and pay statements; partners.
Store	/store/scan	Scan hub.
Shared	/packages/[id], /scan	Public package view; standalone scan.

9. Demo Accounts & Credentials

All demo sign-ins use the @consigntrack.test domain. By convention consignor logins use the password **consignor123** and store logins use **store123**. Keep this page handy for the presentation.

Account	Role	Sign-in email	Password
System Admin	Administrator	admin@consigntrack.test	(your admin password)
Maria's Weaves	Consignor	maria@consigntrack.test	consignor123
Jameer's Warehouse	Consignor	jameer@consigntrack.test	consignor123
Dalisay Handicrafts (new)	Consignor	dalisay@consigntrack.test	consignor123
Kultura - SM Aura	Store manager	kultura@consigntrack.test	store123
Jamie's Warehouse	Store manager	jamie@consigntrack.test	store123
Likha Finds - Glorietta (new)	Store manager	likha@consigntrack.test	store123
Baguio Pasalubong Center	Store manager	baguio@consigntrack.test	store123
Ilocos Craft Corner	Store manager	ilocos@consigntrack.test	store123
Kultura Staff	Store staff	staff@consigntrack.test	(store staff login)

Passwords shown are the demo convention; the administrator can change them. Accounts are admin-provisioned - there is no public sign-up.

New demo dataset: Dalisay Handicrafts -> Likha Finds

A fresh consignor and store were seeded to show a realistic day of operations: an 18% monthly agreement, a shipment dispatched and received about a week ago, a 6-product catalog on the shelf, serialized pieces ready to scan, and roughly a week of sales (including today).

SKU	Product	Price (PHP)	On hand	Sold	Pieces to scan
DAL-0001	Abaca tote bag	850	26	4	24 serialized
DAL-0002	Capiz shell lamp	1,250	10	2	-
DAL-0003	Handwoven placemats (set of 4)	480	34	6	-
DAL-0004	Coconut shell bowl	220	44	16	-
DAL-0005	Beaded statement necklace	360	41	9	40 serialized
DAL-0006	Woven storage basket	640	22	2	-

To demo piece scanning, scan a serial such as DAL-0001-0001 or DAL-0005-0001 at the Likha Finds sales terminal.

10. Conventions, Limitations & Operating Notes

Conventions

- **Currency:** Philippine peso (PHP); amounts display with the peso sign in the app.
- **Time zone:** everything displays in Asia/Manila even though the server stores UTC.
- **Text inputs** auto-capitalize the first letter for names and labels; emails, passwords, codes, and search boxes are left as typed.
- **SKUs & serials:** SKUs are auto-issued per product; serialized pieces are -NNNN.

Known limitations (by design, for the demo)

- The cashless settlement rail is a DEMO: it records a method and reference and drives the two-sided confirmation, but moves no real money and calls no external payment API.
- Serialized pieces are minted at the consignor warehouse; the aggregate ledger governs store on-hand, and selling a piece checks store on-hand rather than tracking each piece's physical location between warehouse and shelf.
- Terminating an account or agreement preserves history (products, ledger, settlements) rather than hard-deleting it, so records stay auditable.

Operating the demo

- Sign in at /login with the credentials in section 9; each role lands in its own workspace.
- A typical walkthrough: (store) receive a shipment -> scan items into a cart -> Record sale -> generate a statement -> pay it; then (consignor) confirm the payment and review what each store holds and owes.
- The app is deployed continuously to Vercel; the live URL is https://consigntrack-zeta.vercel.app.